Sprint 3: APIs and Data

WhosDown: Mobile app that makes pickup sports easier

Chris Lee, Jaimin Fee, Kurt Schimmel (Team 6)

1. API Assessment

WhosDown's data (games, RSVPs, profiles, groups, messages, reviews, notifications) is created by our own users, so we will use **Firebase** as our backend instead of building a custom API. Two outside services supply the rest:

Need	Service
Accounts and guest browsing	Firebase Authentication
App data	Cloud Firestore
Photos and clips	Firebase Storage
Push notifications	Firebase Cloud Messaging
Map and venue search	Apple MapKit (replaces the Google Maps plan from Sprint 1)
Weather for outdoor games	Open-Meteo forecast API (Section 4)

2. Firebase JSON Structure

```
users/{userId}
    private/settings
    friends/{friendId}
    notifications/{notificationId}
venues/{venueId}
games/{gameId}
    rsvps/{userId}
groups/{groupId}
    members/{userId}
conversations/{conversationId}
    messages/{messageId}
reviews/{reviewId}
reports/{reportId}
```

In the examples below, Firestore `Timestamp` values are written as ISO date strings and `GeoPoint` values as `{latitude, longitude}`.

`users/{userId}`: a player's public profile.

```
{
  "displayName": "Jamie Morales",
```

```
  "photoURL": null,
  "bio": "Midfielder. I organize weeknight games and keep them welcoming but
organized.",
  "homeArea": "Oakland / CMU",
  "schoolLine": "CMU '27",
  "position": "Midfielder",
  "isVerified": true,
  "isGuest": false,
  "playStyles": ["social", "competitive"],
  "sportLevels": { "soccer": "intermediate", "tennis": "casual" },
  "stats": {
    "gamesRSVPd": 32,
    "gamesAttended": 31,
    "gamesHosted": 14,
    "mvpCount": 2,
    "playAgainYes": 16,
    "playAgainTotal": 18,
    "feedbackTagCounts": { "reliable": 12, "goodSport": 9 },
    "recentFeedback": ["Clear communication and always shows up."]
  },
  "createdAt": "2026-09-01T18:22:00Z"
}
```


users/{userId}/private/settings: private to the owner.


```
{
  "email": "alex.morgan@cmu.edu",
  "notificationPrefs": {
    "approvals": true, "timeOrLocationChanges": true, "cancellations": true,
    "reviewReminders": true, "newNearbyGames": false, "groupActivity": false,
"messages": true
  },
  "newGameAlert": {
    "sports": ["soccer", "tennis"], "radiusMiles": 3.0,
    "weekdaysOnly": true, "earliestHour": 17, "skillLevels": ["allLevels"]
  },
  "privacy": {
    "gameActivity": "friends", "profileVisibility": "whosDownPlayers",
    "directMessages": "friendsAndTeammates", "friendRequests": "everyone"
  },
  "blockedUserIds": ["u_83jd"],
  "favoriteVenueIds": ["v_cmu_soccer"],
  "pinnedGroupIds": ["g_westside"],
  "searchRadiusMiles": 3.0,
  "fcmTokens": ["<device push token>"]
}
```


users/{userId}/friends/{friendId}


```
{ "displayName": "Mina Kim", "status": "friends", "mutualCount": 4, "since":
"2026-10-02T23:10:00Z" }
```

users/{userId}/notifications/{notificationId}


```
{
  "type": "gameUpdated",
  "title": "Pickup Soccer changed location",
  "body": "Jamie: \"Lights are better on the lower turf.\"",
  "gameId": "gm_7d2k",
  "groupId": null,
  "conversationId": null,
  "change": { "field": "location", "oldValue": "CMU Upper Turf", "newValue":
"CMU Lower Turf", "hostNote": null },
  "expiresAt": null,
  "isRead": false,
  "createdAt": "2026-10-25T22:41:00Z"
}
```


venues/{venueId}


```
{
  "name": "CMU Soccer Field",
  "address": "5000 Forbes Ave, Pittsburgh, PA 15213",
  "location": { "latitude": 40.4433, "longitude": -79.9436 },
  "geohash": "dppnhf",
  "isIndoor": false,
  "surface": "turf",
  "hasLights": true,
  "sports": ["soccer", "football", "ultimate"],
  "createdBy": "u_jamie"
}
```


games/{gameId}


```
{
  "title": "Pickup Soccer",
  "sport": "soccer",
  "description": "Friendly all-level pickup.",
  "notes": "Bring a light + dark shirt. Need 1 goalkeeper.",
  "hostId": "u_jamie",
  "hostName": "Jamie Morales",
  "groupId": null,
  "venueId": "v_cmu_soccer",
  "venueName": "CMU Soccer Field",
  "location": { "latitude": 40.4433, "longitude": -79.9436 },
  "geohash": "dppnhf",
  "isIndoor": false,
  "surface": "turf",
  "hasLights": true,
  "startTime": "2026-10-25T23:00:00Z",
  "endTime": "2026-10-26T00:30:00Z",
  "skillLevel": "allLevels",
  "visibility": "public",
```

```
  "joinType": "approvalRequired",
  "priceCents": 0,
  "playerLimit": 14,
  "waitlistEnabled": true,
  "joinedCount": 12,
  "pendingCount": 2,
  "waitlistCount": 0,
  "attendeePreview": ["AM", "MK", "JB", "TP", "CL", "RS"],
  "fieldReserved": true,
  "reservationWindow": "6:45–8:45 PM",
  "facilityNote": "No field conflict.",
  "mediaURLs": [],
  "status": "scheduled",
  "conversationId": "c_gm_7d2k",
  "createdAt": "2026-10-24T15:02:00Z",
  "updatedAt": "2026-10-25T21:40:00Z"
}
```


games/{gameId}/rsvps/{userId}


```
{
  "userId": "u_taylor",
  "displayName": "Taylor Ross",
  "reliability": 0.86,
  "skillLevel": "beginner",
  "status": "pending",
  "isHost": false,
  "noteToHost": "Happy to play defense or learn in goal.",
  "hostResponse": null,
  "waitlistPosition": null,
  "spotOfferExpiresAt": null,
  "attended": null,
  "personalNote": null,
  "requestedAt": "2026-10-25T20:15:00Z",
  "respondedAt": null
}
```


groups/{groupId}


```
{
  "name": "West Side Pickup Crew",
  "description": "Friendly weeknight soccer. Newcomers welcome.",
  "photoURL": null,
  "sports": ["soccer"],
  "localArea": "Oakland / CMU",
  "visibility": "public",
  "membershipType": "open",
  "ownerId": "u_jamie",
  "memberCount": 48,
  "conversationId": "c_grp_westside",
```

```
  "createdAt": "2026-09-10T00:00:00Z"
}
```

groups/{groupId}/members/{userId}

```
{ "displayName": "Alex Morgan", "role": "member", "joinedAt":
"2026-10-01T19:00:00Z" }
```

conversations/{conversationId}: used for direct, game, and group chats.

```
{
  "type": "game",
  "title": "Pickup Soccer",
  "gameId": "gm_7d2k",
  "groupId": null,
  "participantIds": ["u_jamie", "u_alex", "u_mina"],
  "lastMessageText": "Thanks! Field is the lower turf now.",
  "lastMessageAt": "2026-10-25T22:24:00Z",
  "unreadCounts": { "u_alex": 0, "u_mina": 2 }
}
```

conversations/{conversationId}/messages/{messageId}

```
{ "senderId": "u_mina", "senderName": "Mina Kim", "text": "Can someone bring
an extra ball?", "sentAt": "2026-10-25T22:12:00Z" }
```

reviews/{reviewId}

```
{
  "gameId": "gm_7d2k",
  "reviewerId": "u_alex",
  "revieweeId": "u_jamie",
  "isMVP": true,
  "playAgain": true,
  "tags": ["reliable", "goodSport"],
  "comment": "Clear communication and made sure everyone was involved.",
  "createdAt": "2026-10-26T01:05:00Z"
}
```

reports/{reportId}

```
{
  "reporterId": "u_alex",
  "reportedUserId": "u_83jd",
  "gameId": "gm_7d2k",
  "reason": "noShow",
  "details": "RSVP'd to three games and didn't come to any.",
  "status": "open",
  "createdAt": "2026-10-26T02:00:00Z"
}
```

3. Swift Structs

Each struct matches one Firestore document above and can be decoded directly with `document.data(as:)`. The same code is in `Models.swift`.

```swift
import Foundation
import FirebaseFirestore

// MARK: - Shared enums

enum Sport: String, Codable, CaseIterable {
  case soccer, basketball, tennis, volleyball, pickleball, baseball,
football, ultimate, other
}

enum SkillLevel: String, Codable, CaseIterable {
  case beginner, casual, allLevels, intermediate, competitive
}

enum PlayStyle: String, Codable, CaseIterable {
  case social, competitive, learning
}

enum Visibility: String, Codable {
  case `public`, `private`
}

enum JoinType: String, Codable {
  case open, approvalRequired
}

enum GameStatus: String, Codable {
  case scheduled, cancelled, completed
}

enum RSVPStatus: String, Codable {
  case pending, approved, declined, waitlisted, left, removed
}

enum AudienceSetting: String, Codable {
  case everyone, whosDownPlayers, friendsAndTeammates, friends, nobody
}

enum FeedbackTag: String, Codable, CaseIterable {
  case reliable, goodSport, communicative, welcoming, competitive, helpful
}

// MARK: - users/{userId}
```

```swift
struct AppUser: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var photoURL: String?
  var bio: String
  var homeArea: String
  var schoolLine: String?
  var position: String?
  var isVerified: Bool
  var isGuest: Bool
  var playStyles: [PlayStyle]
  var sportLevels: [String: SkillLevel]
  var stats: PlayerStats
  var createdAt: Date

  var initials: String {
    displayName.split(separator: "
").prefix(2).compactMap(\.first).map(String.init).joined()
  }
}

struct PlayerStats: Codable {
  var gamesRSVPd: Int
  var gamesAttended: Int
  var gamesHosted: Int
  var mvpCount: Int
  var playAgainYes: Int
  var playAgainTotal: Int
  var feedbackTagCounts: [String: Int]
  var recentFeedback: [String]

  var reliability: Double? {
    gamesRSVPd == 0 ? nil : Double(gamesAttended) / Double(gamesRSVPd)
  }
}

// MARK: - users/{userId}/private/settings

struct UserSettings: Codable {
  var email: String
  var notificationPrefs: NotificationPrefs
  var newGameAlert: NewGameAlertRule?
  var privacy: PrivacySettings
  var blockedUserIds: [String]
  var favoriteVenueIds: [String]
  var pinnedGroupIds: [String]
  var searchRadiusMiles: Double
  var fcmTokens: [String]
```

```
}

struct NotificationPrefs: Codable {
  var approvals: Bool
  var timeOrLocationChanges: Bool
  var cancellations: Bool
  var reviewReminders: Bool
  var newNearbyGames: Bool
  var groupActivity: Bool
  var messages: Bool
}

struct NewGameAlertRule: Codable {
  var sports: [Sport]
  var radiusMiles: Double
  var weekdaysOnly: Bool
  var earliestHour: Int
  var skillLevels: [SkillLevel]
}

struct PrivacySettings: Codable {
  var gameActivity: AudienceSetting
  var profileVisibility: AudienceSetting
  var directMessages: AudienceSetting
  var friendRequests: AudienceSetting
}

// MARK: - users/{userId}/friends/{friendId}

enum FriendStatus: String, Codable {
  case requestSent, requestReceived, friends
}

struct Friendship: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var status: FriendStatus
  var mutualCount: Int
  var since: Date
}

// MARK: - users/{userId}/notifications/{notificationId}

enum NotificationType: String, Codable {
  case approved, declined, gameUpdated, gameCancelled, spotOpen, message,
groupActivity, reviewReminder, newGame
}

struct AppNotification: Codable, Identifiable {
```

```
  @DocumentID var id: String?
  var type: NotificationType
  var title: String
  var body: String
  var gameId: String?
  var groupId: String?
  var conversationId: String?
  var change: GameChange?
  var expiresAt: Date?
  var isRead: Bool
  var createdAt: Date
}

struct GameChange: Codable {
  var field: String
  var oldValue: String
  var newValue: String
  var hostNote: String?
}

// MARK: - venues/{venueId}

struct Venue: Codable, Identifiable {
  @DocumentID var id: String?
  var name: String
  var address: String
  var location: GeoPoint
  var geohash: String
  var isIndoor: Bool
  var surface: String?
  var hasLights: Bool
  var sports: [Sport]
  var createdBy: String
}

// MARK: - games/{gameId}

struct Game: Codable, Identifiable {
  @DocumentID var id: String?
  var title: String
  var sport: Sport
  var description: String
  var notes: String?
  var hostId: String
  var hostName: String
  var groupId: String?

  var venueId: String
  var venueName: String
```

```
  var location: GeoPoint
  var geohash: String
  var isIndoor: Bool
  var surface: String?
  var hasLights: Bool

  var startTime: Date
  var endTime: Date

  var skillLevel: SkillLevel
  var visibility: Visibility
  var joinType: JoinType
  var priceCents: Int
  var playerLimit: Int
  var waitlistEnabled: Bool

  var joinedCount: Int
  var pendingCount: Int
  var waitlistCount: Int
  var attendeePreview: [String]

  var fieldReserved: Bool
  var reservationWindow: String?
  var facilityNote: String?
  var mediaURLs: [String]

  var status: GameStatus
  var conversationId: String
  var createdAt: Date
  var updatedAt: Date

  var isFree: Bool { priceCents == 0 }
  var isFull: Bool { joinedCount >= playerLimit }
  var spotsLeft: Int { max(playerLimit - joinedCount, 0) }
}

// MARK: - games/{gameId}/rsvps/{userId}

struct RSVP: Codable, Identifiable {
  @DocumentID var id: String?
  var userId: String
  var displayName: String
  var reliability: Double?
  var skillLevel: SkillLevel?
  var status: RSVPStatus
  var isHost: Bool
  var noteToHost: String?
  var hostResponse: String?
  var waitlistPosition: Int?
```

```swift
  var spotOfferExpiresAt: Date?
  var attended: Bool?
  var personalNote: String?
  var requestedAt: Date
  var respondedAt: Date?
}

// MARK: - groups/{groupId}

enum MembershipType: String, Codable {
  case open, requestToJoin
}

struct PlayGroup: Codable, Identifiable {
  @DocumentID var id: String?
  var name: String
  var description: String
  var photoURL: String?
  var sports: [Sport]
  var localArea: String
  var visibility: Visibility
  var membershipType: MembershipType
  var ownerId: String
  var memberCount: Int
  var conversationId: String
  var createdAt: Date
}

// MARK: - groups/{groupId}/members/{userId}

enum GroupRole: String, Codable {
  case owner, admin, member, pending
}

struct GroupMember: Codable, Identifiable {
  @DocumentID var id: String?
  var displayName: String
  var role: GroupRole
  var joinedAt: Date
}

// MARK: - conversations/{conversationId}

enum ConversationType: String, Codable {
  case direct, game, group
}

struct Conversation: Codable, Identifiable {
  @DocumentID var id: String?
```

```swift
  var type: ConversationType
  var title: String
  var gameId: String?
  var groupId: String?
  var participantIds: [String]
  var lastMessageText: String?
  var lastMessageAt: Date?
  var unreadCounts: [String: Int]
}

// MARK: - conversations/{conversationId}/messages/{messageId}

struct Message: Codable, Identifiable {
  @DocumentID var id: String?
  var senderId: String
  var senderName: String
  var text: String
  var sentAt: Date
}

// MARK: - reviews/{reviewId}

struct Review: Codable, Identifiable {
  @DocumentID var id: String?
  var gameId: String
  var reviewerId: String
  var revieweeId: String
  var isMVP: Bool
  var playAgain: Bool
  var tags: [FeedbackTag]
  var comment: String?
  var createdAt: Date
}

// MARK: - reports/{reportId}

enum ReportReason: String, Codable {
  case harassment, noShow, misleadingEvent, other
}

struct Report: Codable, Identifiable {
  @DocumentID var id: String?
  var reporterId: String
  var reportedUserId: String
  var gameId: String?
  var reason: ReportReason
  var details: String?
  var status: String
```

```
  var createdAt: Date
}
```

4. External API: Open-Meteo Weather

The weather line on Game Detail (H09), Joined Game (H14), and Venue & Details (H18) comes from Open-Meteo, which is free and needs no API key. Our submitted playground, `WhosDownWeather.playground`, requests the forecast for a game's coordinates, decodes the JSON into `Codable` structs, and picks the hour closest to the game's start time.

Request

```
GET
https://api.open-meteo.com/v1/forecast?latitude=40.4433&longitude=-79.9436

&hourly=temperature_2m,precipitation_probability,weather_code,wind_speed_10m

&temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=America/New_York&fo
recast_days=7
```

Response (shortened)

```
{
  "latitude": 40.4349,
  "longitude": -79.93617,
  "timezone": "America/New_York",
  "hourly": {
    "time": ["2026-10-25T00:00", "2026-10-25T01:00"],
    "temperature_2m": [46.9, 44.9],
    "precipitation_probability": [0, 0],
    "weather_code": [0, 0],
    "wind_speed_10m": [4.8, 2.9]
  }
}
```

Playground output

```
Pickup Soccer @ CMU Soccer Field
  56°F · Clear · 0% rain · Outdoor
  Wind 5 mph
7v7 at Schenley @ Schenley Oval
  54°F · Partly cloudy · 0% rain · Outdoor
  Wind 3 mph
```

5. Wireframe Data Mapping

6. List of Features